

AWS API Gateway?

AWS API Gateway is a **fully managed service** that enables developers to:

- Create, publish, maintain, monitor, and secure **APIs at any scale**
 - Build **RESTful, HTTP**, and **WebSocket** APIs
 - Act as a “**front door**” for applications accessing backend services such as:
 - **Lambda functions**
 - **EC2, ECS**
 - **Load balancers**
 - **On-premises servers**
-

Types of APIs in API Gateway

API Type	Description	Use Case Example
REST API	Full-featured API with caching, throttling, auth, etc	Legacy apps, apps requiring fine control
HTTP API	Lightweight, cheaper, faster alternative	Simple microservices, serverless APIs
WebSocket	Bi-directional real-time communication	Chat apps, live dashboards

Key Components of API Gateway

1. Resource (**/users**, **/orders**)

Represents a path in the API.

2. Method (**GET**, **POST**, **PUT**, **DELETE**)

Operation you perform on a resource.

3. Integration

Backend endpoint: Lambda, HTTP URL, VPC link, etc.

4. Stages

Deployment versions of your API (e.g., **dev**, **prod**).

5. Mapping Templates

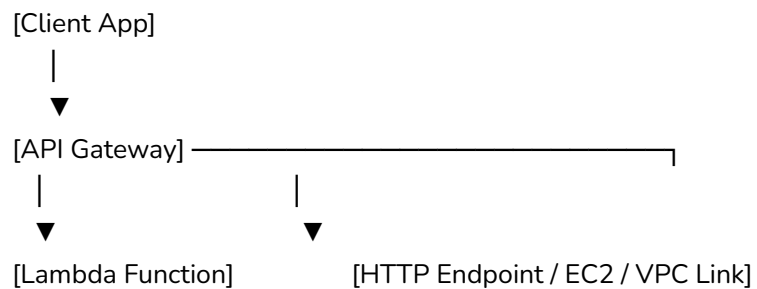
Transform incoming requests or responses using **Velocity Template Language (VTL)**.

6. Authorizers

Secure access with:

- **Cognito User Pools**
- **Lambda authorizer (custom)**
- **IAM-based authentication**

How It Works (Architecture Diagram)



API Gateway processes request → validates → forwards to backend → returns response to client.

Security Features

Feature	Purpose
API Keys	Identify calling apps (basic quota control)
IAM Roles	Fine-grained access with signed requests
Cognito Auth	OAuth/OpenID Connect-based auth
Lambda Authorizer	Custom logic to allow/deny access
Throttling & Quotas	Protect APIs from abuse (rate limits)

Integration Types

Type	Use Case
Lambda Proxy	Serverless backends
HTTP / HTTP Proxy	Forward to any HTTP service (e.g. ALB)
Mock	Simulate responses (for testing)
AWS Service	Direct call to AWS services (e.g. S3, DynamoDB)
VPC Link	Private APIs to backend inside VPC

Features You Should Know (For Architect Role)

- **Caching:** Enable response caching to improve latency (paid feature)
 - **Deployment Stages:** Version control and stage-specific configs
 - **Custom Domain Names:** Use your domain with SSL
 - **Monitoring:** CloudWatch metrics, logging, tracing
 - **SDK Generation:** Auto-generate SDKs for iOS, Android, JavaScript
 - **CORS Configuration:** Allow cross-origin browser access
-

Common Use Cases

Use Case	API Gateway Role
Serverless Web App	Trigger Lambda via REST/HTTP API
Mobile Backend	Expose secure REST API
Microservices Gateway	Central point for all services
IoT Communication	Use WebSocket APIs for devices
Monetized APIs	Enforce API keys & usage plans
Private APIs (inside VPC)	Use VPC Link + NLB or Internal ALB

Example: Lambda + API Gateway

Goal: Build a REST API to fetch user details

1. Create Lambda Function:

- Name: `getUserFunction`
- Logic: Returns user info from DynamoDB

2. Create REST API in API Gateway:

- Resource: `/user`
- Method: `GET`
- Integration: Lambda

3. Deploy API to `prod` stage

4. Test endpoint:

`https://<api-id>.execute-api.<region>.amazonaws.com/prod/user`

Monitoring & Logging

- **CloudWatch Logs:** Enable in method-level settings
 - **Metrics:** 4XX, 5XX errors, Latency, CacheHits
 - **X-Ray Integration:** End-to-end request tracing
-

Pricing Highlights

Metric	Description
Per API Call	Based on request count
Data Transfer Out	Charged like EC2 traffic
Caching	Hourly charges based on cache size
WebSocket Messages	Charged per message sent/received
